Etapa 6 — Projeto técnico: PedeUmTrem

PedeUmTrem · arquitetura, banco de dados, segurança e integrações · 29 de outubro de 2026

1. Arquitetura

O PedeUmTrem é uma única aplicação web (PWA) publicada no Cloudflare Pages que conversa direto com o Supabase, e todas as regras que mexem com dinheiro e status ficam dentro do banco, nunca só no frontend.

- **Frontend:** React, TypeScript e Vite, com PWA (vite-plugin-pwa: manifest e service worker). Quatro áreas com rotas e permissões separadas: cliente, lojista, administrador e entregador (Fase 2).
- **Hospedagem:** Cloudflare Pages ligado ao GitHub. Cada push na branch principal publica em produção, e cada branch gera uma prévia. Variáveis: URL e chave pública (anon) do Supabase.
- **Supabase:** PostgreSQL com RLS, Auth (e-mail e senha, Google), Storage (imagens), Realtime (pedidos novos no painel) e Edge Functions (envio de push).
- **Regras de negócio:** funções do PostgreSQL chamadas pelo frontend (rpc), com SECURITY DEFINER e search_path fixo. Elas calculam preços, comissão e transições de status. Sem Cloudflare Workers.
- **Rotina quinzenal:** pg_cron chama a função de fechamento nos dias 1 e 16.

Fluxo de dados de um pedido: o cliente chama `criar_pedido`, que valida tudo e grava o pedido, os itens e o histórico em uma transação. Um webhook do banco aciona a Edge Function de push. O painel do lojista recebe o pedido por Realtime. O lojista chama `mudar_status_pedido` a cada etapa, e o cliente vê a mudança por Realtime.

2. Modelo de dados

Todos os nomes em português e snake_case. Dinheiro em `numeric(10,2)`. Chaves primárias `uuid`, exceto `pedidos`, que usa número sequencial para ser fácil de citar no WhatsApp.

Pessoas e lojas

Tabela	Campos principais
perfis	id (igual ao usuário do Auth), nome, sobrenome, telefone, papel (cliente, lojista, admin, entregador), situacao (ativo, suspenso), criado_em. O e-mail fica no Auth.
enderecos	id, cliente_id, cep, rua, numero, bairro, complemento, referencia
categorias_estabelecimento	id, nome, descricao, ativa
estabelecimentos	id, dono_id, nome, descricao, categoria_id, telefone, whatsapp, cep, rua, numero, bairro, capa_url, taxa_entrega, areas_atendimento (lista de bairros), tempo_estimado_min, aceita_pix, aceita_cartao, aceita_dinheiro, chave_pix, comissao_percentual (opcional: vazio usa a comissão padrão), modo_entrega (propria, plataforma, ambos), situacao (pendente, aprovado, suspenso), pausado, criado_em
horarios_funcionamento	id, estabelecimento_id, dia_semana, abre, fecha
fechamentos_temporarios	id, estabelecimento_id, inicio, fim, motivo

Cardápio

Tabela	Campos principais
categorias_cardapio	id, estabelecimento_id, nome, ordem. Índice único em (estabelecimento_id, nome em minúsculas) para impedir duplicadas.
produtos	id, estabelecimento_id, categoria_id, nome, descricao, preco (maior ou igual a zero), imagem_url, disponivel, ordem, destaque

Pedidos e dinheiro

Tabela	Campos principais
pedidos	id (sequencial), cliente_id, cliente_nome, estabelecimento_id, status, cópia do endereço (cep, rua, numero, bairro, complemento, referencia), telefone_contato, observacoes, forma_pagamento (pix, cartao_entrega, dinheiro_entrega), valor_troco, subtotal, taxa_entrega, total, percentual_comissao, valor_comissao, tempo_preparo_min, motivo_recusa, chave_idempotencia (única), responsavel_entrega (estabelecimento ou plataforma), criado_em, atualizado_em
itens_pedido	id, pedido_id, produto_id, nome_produto, preco_unitario, quantidade, subtotal, observacoes
historico_pedido	id, pedido_id, status_anterior, status_novo, responsavel_id, observacao, criado_em
comissoes	id, estabelecimento_id, pedido_id (único), valor, situacao (pendente, paga), fechamento_id, gerada_em, paga_em
fechamentos	id, estabelecimento_id, periodo_inicio, periodo_fim, total_comissoes, total_taxas_entrega, vencimento, situacao (aberto, pago, vencido), pago_em
registro_administrativo	id, usuario_id, acao, tabela, registro_id, detalhes (json), criado_em
push_inscricoes	id, usuario_id, endpoint, chaves (json), criado_em

O pedido guarda o nome do cliente e uma cópia do endereço, de modo que o lojista não precisa ler o perfil do cliente e alterar o cardápio ou o endereço depois não muda pedidos antigos (RN10).

Fase 2 (entregadores)

Tabela	Campos principais
entregadores	perfil_id, veiculo, chave_pix, situacao (pendente, aprovado, suspenso), disponivel

Tabela	Campos principais
corridas	id, pedido_id, estabelecimento_id, entregador_id, estado (solicitada, aceita, retirada, entregue, cancelada), tipo (inicial, nova_viagem), motivo, valor_cobrado, valor_repasse, valor_retido, solicitada_em, aceita_em, entregue_em

3. Regras no banco

criar_pedido (chamada pelo cliente). Recebe estabelecimento, itens (produto e quantidade), endereço, telefone, observações, forma de pagamento, troco e a chave de idempotência. Valida que a loja está aprovada, aberta e não pausada, que os produtos são dela e estão disponíveis, que a forma de pagamento é aceita e que o troco é igual ou maior que o total. Calcula subtotal, taxa e total a partir dos preços do banco (o cliente nunca envia valores), grava o percentual de comissão vigente e registra o primeiro histórico. Se a mesma chave de idempotência chegar de novo, devolve o pedido já criado (RNF09).

mudar_status_pedido (lojista, cliente ou administrador). Confere quem está pedindo e se a transição é permitida:

De	Para	Quem
Aguardando confirmação	Aceito (com tempo de preparo) ou Recusado	Lojista
Aguardando confirmação ou Aceito	Cancelado	Cliente
Aceito até Saiu para entrega	Etapa seguinte, podendo pular etapas intermediárias	Lojista
Saiu para entrega	Concluído	Lojista
Aceito até Saiu para entrega	Cancelado, com motivo obrigatório	Lojista

Concluído, Recusado e Cancelado são finais, e nenhum pedido chega a Concluído sem ter sido aceito. Correções excepcionais só pelo administrador, com registro de auditoria. Toda mudança grava uma linha no histórico.

Comissão. Ao ir para Concluído, a função grava `valor_comissao` (subtotal x percentual registrado, arredondado a duas casas) e cria a linha em `comissoes`. Recusados e cancelados não geram comissão. O percentual é alterável pelo administrador: o padrão fica na tabela configuracoes (chave comissao_padrao, valor inicial 10, junto com prazo_pagamento_dias e, na Fase 2, os valores da entrega) e cada loja pode ter um

percentual próprio em estabelecimentos.comissao_percentual. A mudança vale só para pedidos criados depois, porque cada pedido guarda o percentual vigente (RN10). Só o administrador lê e altera configuracoes, e toda alteração vai para o registro administrativo.

Destaques. Um gatilho em `produtos` bloqueia o sétimo destaque da loja e devolve a mensagem de limite atingido.

fechar_quinzena. Roda no dia 1 (período 16 ao fim do mês anterior) e no dia 16 (período 1 a 15). Para cada estabelecimento, cria o fechamento, vincula as comissões pendentes do período, soma os totais e define o vencimento (padrão de 5 dias corridos, configurável). Fechamentos não pagos depois do vencimento passam a vencidos e entram no relatório de inadimplência.

Fase 2. `solicitar_entregador` cria a corrida, e `aceitar_corrida` usa bloqueio de linha para que só o primeiro entregador fique com ela. O repasse (R$ 2,50) e a retenção (R$ 0,50) são gravados no aceite.

4. Segurança e permissões

Regra geral: RLS ligado em todas as tabelas. Clientes e lojistas nunca alteram `pedidos`, `comissoes` ou `fechamentos` diretamente, só pelas funções.

Tabela	Cliente	Lojista	Administrador
perfis	lê e edita o próprio (menos papel e situação)	idem	lê todos e altera situação
estabelecimentos, categorias_cardapio, produtos, horarios	lê lojas aprovadas (leitura pública)	lê e edita as da própria loja (menos situação e percentual de comissão)	tudo
pedidos, itens_pedido, historico_pedido	lê os próprios	lê os da própria loja	lê todos
comissoes, fechamentos	sem acesso	lê os da própria loja	tudo
registro_administrativo	sem acesso	sem acesso	só leitura (a gravação é feita pelas funções)

Tabela	Cliente	Lojista	Administrador
push_inscricoes	cria e apaga as próprias	idem	sem acesso

- **Papéis:** `papel` não pode ser alterado pelo próprio usuário. Funções auxiliares (`eh_admin()`, `dono_da_loja(id)`) são usadas nas políticas para evitar leitura recursiva de `perfis`.
- **Autenticação:** senha com no mínimo 8 caracteres e exigência de letras e dígitos, configurada no Auth, com a mesma regra no formulário. Nada de hash de senha escrito à mão.
- **Segredos:** a chave `service_role` e a chave privada do push existem só nas Edge Functions, nunca no frontend nem em logs.
- **Storage:** buckets públicos para leitura de capas e fotos. Escrita só do dono, dentro da pasta com o id da própria loja.
- **Privacidade (LGPD):** endereço e telefone só aparecem para quem participa do pedido (cliente, loja e administrador). Não se guardam dados de cartão.
- **Testes de segurança:** roteiro com os três perfis tentando ler ou alterar dados de outra loja, mudar o próprio papel, forjar valores e pular transições de status. Todos devem falhar.

5. Integrações

- **Consulta de CEP (ViaCEP):** chamada direta do navegador. Se falhar, o cliente preenche rua e bairro à mão.
- **Push:** service worker, chaves VAPID e uma Edge Function acionada por webhook quando um pedido é criado ou muda de status. No iPhone, só funciona com a PWA instalada na tela inicial.
- **Tempo real:** o painel do lojista assina os pedidos da própria loja, e o cliente assina o próprio pedido. A RLS vale também para o Realtime.
- **WhatsApp:** link `wa.me` com o resumo do pedido montado no frontend, sem API. O resumo inclui itens, valores, pagamento (com troco, se houver) e endereço.
- **Imagens:** compressão e conversão para WebP no navegador (800 x 800 para produtos, 1200 x 675 para capa, até 200 KB e 300 KB), depois envio ao Storage.
- **Fase 2:** push também para entregadores. Nenhuma integração nova.

6. Código, ambientes e qualidade

- **Bibliotecas recomendadas:** React Router, TanStack Query, React Hook Form com Zod (validação), Tailwind CSS, supabase-js e tipos gerados do banco (`supabase gen types`).
- **Dinheiro no frontend:** valores em centavos inteiros, nunca decimal. O texto digitado com vírgula (12,50) é convertido antes de enviar.
- **Validação igual nos dois lados:** os esquemas Zod espelham as regras do banco (restrições, `check` e funções), e as mensagens identificam o campo e o motivo.
- **Estrutura do repositório:** `src/` com uma pasta por área (cliente, lojista, admin, entregador) e `shared/` (componentes, bibliotecas, hooks), mais `supabase/` com `migrations/`, `functions/` e `seed.sql`.
- **Ambientes:** dois projetos Supabase (desenvolvimento e produção) e Supabase CLI local. Toda mudança de banco é uma migração versionada no Git, aplicada primeiro em desenvolvimento.
- **Testes:** Vitest para cálculos e validações, testes do banco (pgTAP) para funções e RLS, e Playwright para os dois fluxos principais (checkout e painel do lojista).
- **Monitoramento:** erros do frontend em um serviço gratuito (por exemplo, Sentry) e logs do Supabase para o banco e as funções.

7. Pontos que assumi nesta etapa

Decidi estes pontos para não travar a etapa. Se algum não servir, é só dizer:

- O lojista se cadastra pelo app e cria a loja como pendente, até o administrador aprovar.
- A área de atendimento é uma lista de bairros por loja, e o checkout avisa se o bairro não é atendido.
- O lojista pode pular etapas intermediárias de status (por exemplo, de Aceito direto para Saiu para entrega), mas não concluir pedido que não foi aceito.
- O fechamento quinzenal é automático, e o administrador só registra o pagamento.